

TJMJ: A Full-Stack Web-Based Taojiang Classic Wang Mahjong Platform for Entertainment & Education

全栈技术手册 v2.0

JaiJaiC* TJMJ Development Team
Taojiang, Hunan, China
{jaijai.c}@tjmj.dev

2026 年 6 月 25 日

Abstract 桃江经典王麻将 (Taojiang Jingdian Wang Majiang, TJMJ) 是一款面向湖南桃江地区麻将爱好者的全栈网页麻将游戏平台。本项目完整实现了桃江地方麻将的核心规则——包括打筛扳王（癞子）机制、扎鸟翻倍、门子累加番、组合胡法等复杂计分体系，并提供单机模式、四人联机对战、观战模式三大玩法。技术架构采用 Vue 3 Composition API + Vite 8 前端、Node.js + ws WebSocket 服务端、WebRTC P2P 实时语音（含 Twilio TURN 中继），前端通过 GitHub Pages 托管、后端通过 ngrok 公网穿透。全文共八节，系统阐述游戏规则数学模型、前后端全栈架构（含完整通信协议与数据流）、核心算法实现（递归回溯胡牌判定、听牌检测、NPC 启发式策略）、多平台设备兼容性方案（iOS/Android/桌面端）、语音系统设计以及未来 Mahjong Agent 训练方向。本项目完全开源（MIT License），仅供家庭娱乐与学术研究使用。

目录

1	Introduction	3
2	Game Rules & Mathematical Formulation	3
2.1	Tile Set & Terminology	3
2.2	Dice Rolling & Wall Breaking	3
2.3	Basic Actions & Priority	4
2.4	Winning Patterns (Hu Types)	4
2.5	Scoring & ZhaNiao (扎鸟)	5
3	System Architecture	6
3.1	Overview	6
3.2	Technology Stack	6

*Correspondence: <https://github.com/JaijaiC/TJMJ>

3.3	Project Structure	6
3.4	Communication Protocol	7
3.5	Server Architecture: GameRoom Finite State Machine	8
3.6	Client Network Layer	9
3.7	Deployment Topology	9
4	Core Algorithms	10
4.1	Hu Detection Algorithm (胡牌判定)	10
4.2	Ting Detection Algorithm (听牌检测)	10
4.3	NPC Strategy Agent (AI 出牌决策)	11
4.4	Voice System	11
4.5	WebRTC Voice Architecture	11
5	Features & User Interface	12
5.1	Game Modes	12
5.2	Interactive Features	12
6	Device Compatibility	12
6.1	Mobile Device Handling	12
6.2	Known Remaining Issues	13
7	Development & Deployment	13
7.1	Development Environment	13
7.2	Startup Commands	14
7.3	Data Flow: A Complete Game Round	14
	Future Work	15
	Code Availability	16
	Disclaimer	16
	Acknowledgments	16

1 Introduction

麻将 (Mahjong) 作为中国传统博弈游戏的代表，在湖南桃江地区形成了独具特色的地方变种——桃江经典王麻将。其核心特征包括：108 张万/筒/条字牌（不含风牌与花牌）、“王”（癞子/万能牌）机制、打筛扳王确定癞子、扎鸟翻倍计分、门子累加番型等复杂规则，与标准国标麻将或四川血战麻将存在显著差异。

目前市面上仅有闲逸等少数 App 提供益阳麻将玩法，但其存在以下不足：(1) 需要注册登录，操作繁琐；(2) 规则与桃江本地玩法不完全一致；(3) 缺乏拖拽手牌、实时语音、文字弹幕等功能；(4) 无法训练 AI 对弈智能体。

为此，我们设计并开发了 TJMJ——一款与桃江麻将规则高度一致、无需安装、点击链接即可进入的网页麻将平台。本文第 2 节详述游戏规则与数学模型；第 3 节展示全栈系统架构与通信协议；第 4 节剖析核心算法实现；第 5 节介绍联动功能（语音、头像、音效）；第 6 节分析多平台兼容性方案；第 7 节总结已解决与仍存在的问题；第 8 节记录开发方法与部署流程。

项目规模：总计约 4200 行核心代码，其中前端主文件 `App.vue` 2600 行、服务端 `GameRoom.js` 494 行、WebSocket 客户端 224 行、AI 策略 227 行、胡牌引擎 208 行、规则检查 142 行、语音模块 100 行。

2 Game Rules & Mathematical Formulation

2.1 Tile Set & Terminology

桃江麻将使用精简牌组——共 108 张牌，三种花色各 36 张：

- 万 (Wan, W): 一万 ~ 九万，各 4 张 (11–19)
- 条 (Tiao, T): 一条 ~ 九条，各 4 张 (21–29)
- 筒 (Tong, B): 一筒 ~ 九筒，各 4 张 (31–39)

编码约定：花色十位数 (1= 万, 2= 条, 3= 筒) + 数字个位数。例：38 表示八筒，15 表示五万。

术语表：

- 王 ($Wang$): 癞子/万能牌，可替代任意牌使用。由开局“打筛扳王”确定。
- 地 (Di): 扳开的那张牌本身，用于判定地胡。
- 将 ($Jiang$): 数字为 2、5、8 的牌（任意花色），共 36 张。胡牌必需品。
- 一句话 ($YiJuHua$): 三张牌构成 AAA（刻子）或 ABC（顺子，同花色连续）。
- 听牌 ($Ting$): 13 张手牌差任意一张即可胡牌的状态。
- 扎鸟 ($ZhaNiao$): 胡牌后翻下一张牌，根据数字确定输赢翻倍。
- 门子 ($MenZi$): 特殊胡牌类型（碰碰胡、清一色等），采用累加番规则。

2.2 Dice Rolling & Wall Breaking

开局由庄家掷两枚骰子，骰子点数记为 d_1, d_2 ($d_1 \leq d_2$, $d_i \in \{1, \dots, 6\}$)。设 $S = d_1 + d_2$ 。

定庄与数墙：庄家位置编号为 P_0 ，逆时针依次为下家 P_1 、对家 P_2 、上家 P_3 。从庄家开始数到第 S 家（庄家 = 1, 5, 9），该玩家面前的牌墙即为起手墙。

开牌位置：在该墙右手边开始往左数，跳过 d_1 （较小骰数）叠，从下一叠开始抓牌。

抓牌顺序：庄家起抓，每人每次取一码（两叠=4张），共三轮（ $3 \times 4 = 12$ 张/人）。然后“收庄”每人各补1张（各得13张），庄家再补1张（14张），游戏开始。

扳王规则：从起手墙对应玩家的下家墙左手边开始往右数，到第 d_2 （较大骰数）叠处扳开最上面一张牌，此牌即为“地”（ T_d ）， $T_d + 1$ （模10循环，花色不变）即为“王”/癞子：

$$T_w = \begin{cases} \lfloor T_d/10 \rfloor \times 10 + 1, & \text{if } T_d \bmod 10 = 9 \\ T_d + 1, & \text{otherwise} \end{cases} \quad (1)$$

2.3 Basic Actions & Priority

玩家回合可执行以下操作，优先级由高到低：

1. **胡 (Hu)** 一点炮/自摸，直接获胜（最高优先级）
2. **碰 (Pong) / 杠 (Gang)** 一三张相同牌碰之；四张相同可开杠
3. **吃 (Chi)** 一仅能吃上家打出的牌，构成顺子

优先级规则：点炮（胡）> 碰（杠）> 吃。联机模式中，若同时可碰和可吃，两个按钮同时亮起供玩家选择。抢杠规则：若明杠的牌恰好是别家要胡的牌，直接判杠家全赔。

吃 (Chi)

只能吃上家打出的牌，且必须与手中两张牌构成同花色顺子。例：上家打4万，手中有(2,3)或(3,5)或(5,6)万即可吃。

碰 (Pong)

任意玩家打出一张牌，若手中有两张相同即可碰。碰后三张亮出（副露），手牌数减2。

杠 (Gang)

分明杠（他人打出，手中有三张）和暗杠（手中四张，未亮出）。开杠后重新掷骰子，从牌墙末尾倒数摸三张。若摸到能胡的牌直接胡（杠上花），否则由下家开始依次判定能否胡此三张。

2.4 Winning Patterns (Hu Types)

胡牌的基本结构为：一对将 (**Jiang**) + 四句话 (**Sentence**)，其中“一句话”可以是刻子 (AAA) 或顺子 (ABC)。

表 1 列出了所有胡法类型及其番数。门子胡法采用累加番规则。

表 1: 胡法类型与番数对照表

类型	条件	番数
平胡	有王，基本构成	1 番
硬庄	无王，基本构成	2 番
碰碰胡	四句话全为刻子 AAA	2 番
清一色	手牌全部同花色	2 番
七小对	14 张组成 7 个对子	2 番
将将胡	14 张全部为 2/5/8	2 番
起手胡	庄家 14 张首轮直接胡	2 番
开杠	明杠/暗杠后摸三张胡牌	2 番
地胡	摸到三张“地”即可胡	2 番
天胡	手牌有 3 个王	3 番
天天胡	手牌有 4 个王	5 番
黑天胡	首轮无王、无将、无一句话	2 番
地胡带拖	三张地不胡，当一句话用	3 番

组合胡法

当多种门子同时满足时，番数累乘。通用计算公式：

$$S_{\text{自摸}} = \left(\sum \text{门子倍数} \right) \times F_{\text{扎鸟}} \times F_{\text{杠}} \quad (2)$$

实例：

- 天胡七小对全中： $(3 + 2) \times 2 = 10$ 番
- 天胡杠上花全中： $3 \times 2 \times 2 = 12$ 番
- 硬庄将将胡碰碰胡天胡杠上花全中： $(2 + 2 + 2 + 2) \times 2 \times 2 = 32$ 番

2.5 Scoring & ZhaNiao (扎鸟)

胡牌分为自摸（自己摸到胡牌）和抓炮（他人打出胡牌）两种方式。

自摸计分：胡牌后进行扎鸟：翻开本该下家摸的那张牌 (T_z)，根据 $T_z \bmod 10$ 判定：

- 1, 5, 9（全中）：三家输分均翻倍
- 2, 6：下家输分翻倍
- 3, 7：对家输分翻倍
- 4, 8：上家输分翻倍

设基础分 $B = 3$ （平胡 1 番 $\times$ 底分 3），番数 M ，扎鸟系数 $Z \in \{1, 2\}$ ：赢家得分 = 输家各支付 $B \times M \times Z$ （扎中者）或 $B \times M$ （未扎中者）。

核心规则：有王只能自摸（不可抓炮），无王可以抓炮。这一规则使“硬庄”（无王胡牌）更具策略价值。

3 System Architecture

3.1 Overview

TJMJ 采用三层 B/S 架构。前端 Vue 3 SPA 部署于 GitHub Pages，服务端 Node.js WebSocket 运行于本地并通过 ngrok 暴露公网端点，WebRTC 语音为纯 P2P 网状拓扑。

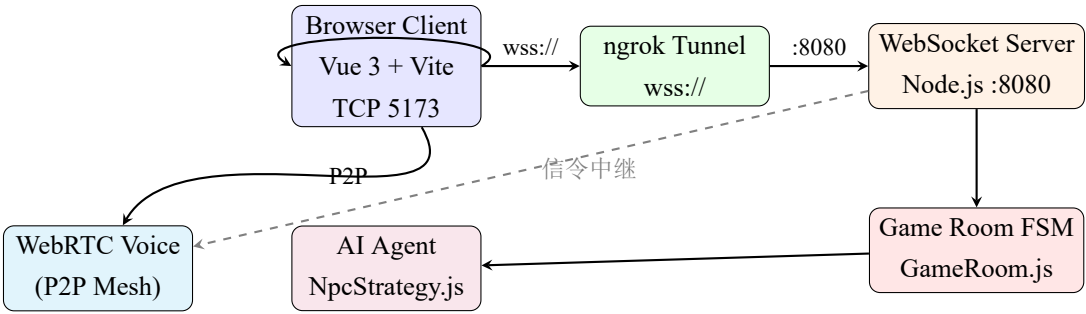


图 1: 系统架构拓扑图

3.2 Technology Stack

表 2: 技术栈总览

层	技术	作用
前端框架	Vue 3.5 (Composition API)	响应式 UI
构建工具	Vite 8 + Rolldown	打包/HMR
前端部署	GitHub Pages	静态托管
服务端运行时	Node.js	无框架纯 JS 服务
服务端通信	ws (v8.18)	WebSocket
服务端部署	ngrok	TCP 隧道 (公网)
实时语音	WebRTC (P2P 网状)	4 人通话
NAT 穿透	Twilio TURN (10GB/月)	移动网络穿透
语音合成	Web Speech API + 预录 mp3	牌名播报
CI/CD	GitHub Actions	PR build 检查

3.3 Project Structure

完整的项目文件树及其职责：

```
1 TJMJ/  
2   frontend/                                # Vue 3 SPA  
3     src/  
4       App.vue                             # 主组件 (2602行: UI+逻辑+样式)  
5       main.js                             # Vue 入口: createApp(App).mount('#app')  
6       style.css                           # 全局样式 (iOS 安全区等)  
7     core/  
8       constants.js                        # 花色枚举、258 判定、王计算  
9       HuCalculator.js                     # 递归回溯胡牌判定引擎 (208行)  
10      RuleChecker.js                      # 吃/碰/杠/听牌合法性 (142行)
```

```

11     ai/
12         NpcStrategy.js           # NPC 启发式出牌策略 (227行)
13     network/
14         client.js               # WebSocket 客户端 (224行: 重连/心跳/节流)
15     utils/
16         mjLogic.js             # 洗牌算法 + 牌墙布局
17         speech.js              # 语音播报 (预录音优先 → TTS 降级)
18     components/
19         HelloWorld.vue          # Vite 脚手架模板 (未使用)
20     public/
21         images/                 # 牌面素材(1-9万条简 + 3D背景)
22         sfx/                   # 预录制音效 (.gitkeep 预留)
23         docs/                  # PDF教程 + 演示视频
24         *.mp3                  # BGM 背景音乐 (1-13.mp3)
25     index.html
26     vite.config.js             # base: '/TJMJ/' (GitHub Pages)
27     package.json               # vue 3.5 + vite 8.0 + gh-pages 6
28     server/                   # Node.js 游戏服务器
29         index.js               # WebSocket 入口 (247行)
30         GameRoom.js           # 房间状态机 (494行)
31         game/                 # 游戏逻辑 (前端镜像, 服务端权威校验)
32             constants.js / HuCalculator.js / RuleChecker.js / mjLogic.js
33     package.json               # ws + uuid
34     backend/                   # 早期 Socket.io 版本 (已废弃)
35     other/                     # 本地工具 (ngrok.exe + 演示视频)
36     .github/workflows/deploy.yml # GitHub Actions CI (build 检查)
37     start-all.bat             # 一键启动 (3 窗口)
38     readme.md / ARCHITECTURE.md / .gitignore

```

代码量统计:

表 3: 核心文件代码行数

文件	行数
frontend/src/App.vue	2602
server/GameRoom.js	494
server/index.js	247
frontend/src/ai/NpcStrategy.js	227
frontend/src/network/client.js	224
frontend/src/core/HuCalculator.js	208
frontend/src/core/RuleChecker.js	142
frontend/src/utils/speech.js	100
合计	4244

3.4 Communication Protocol

客户端与服务器通过单一 WebSocket 长连接进行 JSON 消息通信。完整协议定义如下:

表 4: WebSocket 消息协议（完整版）

type	方向	说明
create_room	C→S	创建新房间（可选 4 位指定号码）
join_room	C→S	加入房间（8888= 测试）
ready	C→S	准备就绪（4 人满自动开局）
ready_next	C→S	确认继续（4 人全确认开新局）
discard	C→S	出牌（携带牌值 tile）
draw	C→S	摸牌
action	C→S	吃/碰/杠/胡操作（携带 action + combo）
pass	C→S	过牌
chat	C↔S	文字弹幕（携带 text + fromName）
emoji	C↔S	表情（携带 icon + target）
webrtc_offer/answer/ice	C↔S	WebRTC 信令中继（不解析 data）
update_avatar	C→S	头像更新（携带 avatar 数据）
get_turn	C→S	请求 TURN 凭据
join_spectate	C→S	加入观战
ping	C→S	心跳检测
room_created/joined	S→C	房间创建/加入成功
room_players	S→C	玩家列表更新（含昵称/头像/状态）
game_start	S→C	开局（每人收到自己的 hand、wangTile、dice）
game_state	S→C	实时状态同步（弃牌/副露/剩余牌数/当前回合）
action_prompt	S→C	请求玩家操作（含 canHu/Peng/Gang/Chi）
drew_tile	S→C	摸到某张牌
round_end	S→C	本局结束（含四家亮牌）
pong	S→C	心跳回复
turn_credentials	S→C	Twilio TURN 凭据（动态生成）
avatar_updated	S→C	头像已同步
game_end	S→C	16 局终局

3.5 Server Architecture: GameRoom Finite State Machine

GameRoom.js 是整个联机游戏的核心——它是一个有限状态机，管理从等待到结算的完整生命周期。

状态转换：

LOBBY → (4人满 + 全部 ready) → PLAYING

PLAYING → (有人胡牌 / 流局) → SETTLEMENT

SETTLEMENT → (4人全部确认继续) → PLAYING（下一局）

SETTLEMENT → (第16局结束) → LOBBY

核心方法：

表 5: GameRoom 核心方法

方法	职责
<code>startGame()</code>	洗牌、发牌 (53 张)、扳王、广播 <code>game_start</code>
<code>handleDiscard(p, tile)</code>	出牌处理: 移除手牌 → 广播 → <code>checkIntercepts</code>
<code>handleDraw(p)</code>	摸牌处理 → 自摸检测
<code>checkIntercepts(p, tile)</code>	检查 3 家能否吃碰杠胡 → 通知相关玩家
<code>resolveActions()</code>	收集响应 → 优先级排序 (胡 > 杠 > 碰 > 吃) → 执行
<code>nextTurn()</code>	切换回合 → 发新牌 → 通知摸牌
<code>endRound(winner)</code>	结算: 赢家坐庄、亮牌、等待确认
<code>playerReadyForNext(p)</code>	确认计数, 4 人满后开新局
<code>broadcastPublic()</code>	向所有玩家发送 <code>game_state</code>

关键设计:

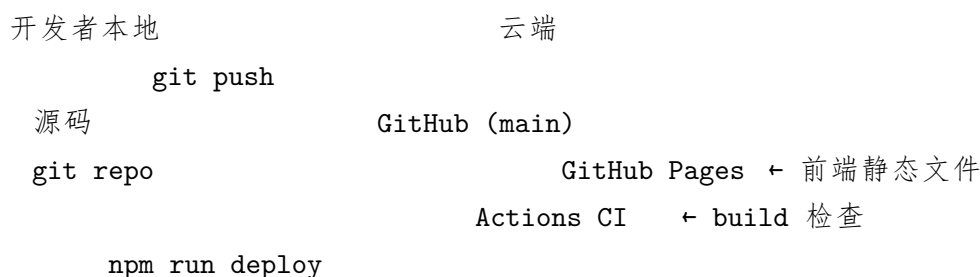
- **双端校验:** `frontend/core/` 和 `server/game/` 各有一份相同的游戏逻辑 (`HuCalculator`, `RuleChecker`, `mjLogic`)。单机模式走前端, 联机模式前端仅做本地预览、服务端做权威判定。
- **超时保护:** `checkIntercepts` 设置 `pendingActions` 后会启动 10 秒超时。若某玩家断线未响应, 超时自动强制清除并调用 `nextTurn()`, 防止游戏永久卡死。
- **手牌不排序:** 服务端发牌和摸牌均不调用 `.sort()`, 保留发牌自然顺序。客户端通过计数差集算法智能合并——仅增删变更的牌, 保持玩家手动拖拽顺序不变。
- **防重复抓牌:** `physicalDraw()` 自动跳过 `diIndex` (地牌位置), 确保扳开的牌不会被摸走。

3.6 Client Network Layer

`client.js` 实现了生产级的 WebSocket 网络层:

- **自动重连:** 指数退避策略 (`1s → 2s → 4s → ... → 128s`), 最多 8 次。重连成功后自动重新加入原房间 (利用 URL 参数 `?auto_join=` 保存房间号和昵称)。
- **心跳检测:** 每 10 秒发 ping, 收到 pong 后计算往返延迟, 更新游戏界面左上角 4 格信号图标 (<100ms 满格, <400ms 两格)。
- **消息节流:** 游戏操作 400ms 节流防止高频崩溃; WebRTC 信令和聊天不节流。
- **缓冲队列:** 连接建立前的消息先入队列, `onopen` 后批量冲刷。
- **URL 持久化:** 创建/加入房间后自动将房间号和昵称写入 URL 参数, 支持页面刷新后自动重连回当前对局。

3.7 Deployment Topology



```
(gh-pages -d dist)
```

```
start-all.bat
```

```
Window 1: server → :8080
```

```
Window 2: frontend → :5173
```

```
Window 3: ngrok http 8080
```

玩家访问 <https://jaijaic.github.io/TJMJ/>

WebSocket → ngrok → server :8080

WebRTC Voice → P2P mesh + Twilio TURN (10GB/月)

4 Core Algorithms

4.1 Hu Detection Algorithm (胡牌判定)

胡牌判定是麻将引擎最核心的算法模块，对应文件 `HuCalculator.js`（前端和 `server/game/` 各一份镜像）。算法采用递归回溯 + 贪心剪枝策略：

1. 预处理：分离王牌与普通牌，统计 n_w （王数量）和 n_d （地数量）
2. 特殊拦截：首轮 14 张检查黑天胡（无王、无将、无一句话）
3. 顶级判定：4 王 → 天天胡, 3 王 → 天胡, 3 地 → 地胡
4. 常规判定：调用 `checkNormalHu()` 递归去除刻子/顺子
5. 七小对判定：调用 `check7Pairs()` 贪心配对
6. 门子判定：将将胡 → 清一色 → 碰碰胡 → 硬庄 → 软庄，依序匹配

`checkNormalHu(normalTiles, wangCount)` 的递归逻辑：

1. 若王充足，补齐对子（将）后递归去除句子
2. 每次选择最小的牌 $t_{\min}$ ，尝试三种消除：
 - 刻子： $\{t, t, t\}$ 三张相同
 - 顺子： $\{t, t+1, t+2\}$ 三张连续（同花色）
 - 对子： $\{t, t\}$ 作为将（仅当将未确定时）
3. 若普通牌无法匹配，消耗 1 个王替代，继续递归
4. 全部消除成功 → 胡牌；任何分支失败 → 回溯

计算复杂度：最坏情况 $O(3^k)$ ， k 为递归深度，通常 $k \leq 4$ ，单次判定 $< 1\text{ms}$ 。

4.2 Ting Detection Algorithm (听牌检测)

听牌检测用于判断 13 张手牌是否差一张即可胡牌，以及开杠后是否能听牌。对应 `RuleChecker.isTing()`：

1. 手牌数若不为 $3n+1$ ，直接返回 `false`
2. 枚举所有可能的 34 种牌（效率优化：仅搜索手牌已有花色 ± 2 范围）
3. 对每种候选牌 t ，模拟加入手牌后调用 `HuCalculator.checkHu()`
4. 若任意候选能胡 → 听牌 `true`；否则 `false`

计算复杂度 $O(34 \times 3^k)$ ，实战 $< 1\text{ms}$ 。

4.3 NPC Strategy Agent (AI 出牌决策)

NpcStrategy.js (227 行) 实现启发式决策:

1. 听牌优先: 遍历每张手牌, 模拟丢弃后检查是否听牌 (`findBestDiscardToTing()`)。若能直接听牌, 优先打出该牌。
2. 价值排序: 若未听牌, 按优先级评估手牌价值 (`chooseWorstTile()`):
 - 孤张 (无相邻牌、非将、非王) → 最优先丢弃
 - 边张/坎张 (只能等 1 张进张) → 次优先
 - 搭子 (差 1 张成顺子/刻子) → 保留
 - 将对 (2/5/8 对子) → 高价值保留
 - 王牌 → 绝不丢弃
3. 安全出牌: 参考弃牌池, 避免打出别家可能胡的炮牌
4. 吃碰决策: `shouldPeng()` / `shouldChi()` 评估手牌改善程度

未来方向: 升级为 DRL (深度强化学习) + MCTS (蒙特卡洛树搜索) 的自我对弈训练智能体。

4.4 Voice System

speech.js (100 行) 实现双轨语音播报:

1. 预录制优先: 检测 `public/sfx/` 下是否存在对应 mp3 文件 (如 `tile_15.mp3` → “五万”), 存在则直接播放 Audio 元素
2. TTS 降级: 文件缺失时使用浏览器 SpeechSynthesis API 合成语音
3. Web Audio 合成: “咚” 出牌音由 OscillatorNode 实时合成 (800Hz → 200Hz 扫描, 150ms)
音效文件清单 (需录制 34 个文件放入 `public/sfx/`): 27 个牌名 (`tile_11.mp3`–`tile_39.mp3`) + 7 个动作词 (`chi.mp3`, `peng.mp3`, `gang.mp3`, `hu.mp3`, `zimo.mp3`, `win.mp3`, `pass.mp3`)。

4.5 WebRTC Voice Architecture

联机语音采用 **P2P 网状拓扑 (Mesh)**: 每个玩家与房间内其他 3 人各建立一条 `RTCPeerConnection`。信令 (`offer/answer/ICE candidate`) 通过 WebSocket 服务端中继。

- **STUN/TURN**: Google STUN × 2 + Twilio TURN (10GB/月免费额度)。TURN 凭据由服务端 HMAC-SHA1 动态生成, 24 小时有效, 每小时刷新。
- 音频约束: 使用 `echoCancellation`, `noiseSuppression`, `autoGainControl` 三项降噪。
- 增量连接: 不因玩家列表变化而拆毁已有连接; 仅在玩家加入/离开时增量创建/移除对应 `PeerConnection`。
- 音轨替换: 麦克风重新开启时, 遍历已有连接替换 `senders` 音轨。
- iOS 兼容: `AudioContext` 创建后检测 `suspended` 状态并显式 `resume()`。

5 Features & User Interface

5.1 Game Modes

平台提供三种游戏模式，均通过 `enterGame(mode)` 入口统一调度：

1. **单机模式**：玩家 vs 3 个 NPC AI。适合练习、测试规则。由客户端 `startRound()` 完全控制游戏流程。
 2. **联机模式**：4 人实时对战。创建/加入 4 位数字房间号。服务端 `GameRoom` 管理状态，客户端仅做本地预览。
 3. **观战模式**：旁观正在进行的对局。接收 `game_state` 同步渲染，可选择观看任意一方视角。
- 模式切换数据流：

`isInMenu=true`

点击"单机模式" → `enterGame('single')`

`handleReady()` → `startRound()` → 本地游戏循环

点击"联机模式" → `enterGame('multi')`

`createRoom()` / `joinRoom()` → `WebSocket` → 等待4人 → 服务器开局

点击"观战模式" → `enterGame('spectate')`

`joinSpectate()` → 接收 `game_state` 并显示

点击"其他玩法" → 已禁用

5.2 Interactive Features

- **拖拽手牌**：HTML5 Drag & Drop + 触屏 Touch Events 双支持。手牌稳定 ID key 防止 Vue VNode 重排。所有游戏操作（出牌/吃/碰/杠）均同步更新 `handTileIds` 数组保持顺序一致。
- **头像系统**：点击自己头像弹出 12 格预设选择面板 + 拍照/上传自定义图片（`FileReader` → `base64 data URL`）。联机模式下通过 `update_avatar` 消息实时同步给其他三人。头像存储于 `localStorage` 持久化。
- **表情互动**：5 种表情（□□□□□）点击发送，飞行动画效果。□ 真转账 1 分。
- **文字弹幕**：右上角聊天按钮，消息从右向左滚动，兼容桌面和移动端。
- **BGM 系统**：首页 2 首交替播放 + 游戏内 9-13 首随机不重复队列 + O 叔钢琴模式（7 首）。统一音量压缩（`Web Audio DynamicsCompressor`）。
- **实时语音**：WebRTC P2P 网状通话，Twilio TURN 中继。音量实时显示条形波动。
- **语音播报**：出牌时自动播报牌名（“五万”“碰”等）。

6 Device Compatibility

6.1 Mobile Device Handling

本项目已实现以下跨平台兼容措施：

表 6: 兼容性修复清单

问题	解决方案
手机 NAT 穿透失败	Twilio TURN (10GB/月) + Google STUN ×2
iOS Safari click 不触发	<div> → <button> 替换非交互元素
iOS AudioContext 静音	检测 <code>suspended</code> 状态并 <code>resume()</code>
iOS 双击缩放	@touchend handler 中 <code>preventDefault()</code>
Android 地址栏跳动	100dvh 替代 100vh (带 fallback)
触屏慢点击漏检	轻点阈值从 300ms 扩大到 500ms
WebSocket 断线	指数退避重连 (1s→128s, 最多 8 次)
桌面端 fixed 弹窗偏移	弹窗组件移出 <code>game-wrapper</code> 避免 transform 影响
联机操作卡死	服务端 10 秒超时强制清除 <code>pendingActions</code>
麦克风权限拒绝	特征检测 + 用户提示

6.2 Known Remaining Issues

1. 语音偶发断续: P2P 网状拓扑在 3+ 人时复杂度 $O(n^2)$, 高丢包环境下部分连接可能掉线。
长期方案: 引入 SFU (Selective Forwarding Unit) 中转。
2. 桌面端 CSS 缩放: `updateGameScale()` 对宽屏/竖屏适配仍有边界情况。
3. 手牌拖拽范围: 触屏拖拽时 DOM 查询 `getBoundingClientRect()` 在 CSS Transform 缩放后可能返回不准确坐标。
4. 音效文件缺失: 34 个预录制 mp3 文件尚未录制, 当前全部降级为 TTS。
5. 观战模式功能有限: 仅支持选择一方视角, 不支持自动切换。

7 Development & Deployment

7.1 Development Environment

表 7: 开发环境与依赖

组件	版本/技术
前端框架	Vue 3.5 (Composition API + <script setup>)
构建工具	Vite 8.0 + Rolldown
服务端	Node.js + ws 8.18
语音	Web Speech API + WebRTC + 预录 mp3
TURN 中继	Twilio TURN (10GB/月, HMAC-SHA1 动态凭据)
内网穿透	ngrok (前端 localhost 5173, 后端 8080)
版本管理	Git + GitHub
前端部署	GitHub Pages (gh-pages -d dist)
CI	GitHub Actions (PR 触发 build 检查)
Node 依赖	vue, vite, @vitejs/plugin-vue, gh-pages ws, uuid

7.2 Startup Commands

一键启动 (Windows):

```
1 .\start-all.bat
2 # 依次启动 3 个窗口:
3 #   [1] Game Server   WebSocket :8080
4 #   [2] Frontend      Vite :5173
5 #   [3] ngrok Tunnel  (forward 8080, optional)
```

手动启动:

```
1 # Terminal 1 - Game Server
2 cd server && npm run dev
3
4 # Terminal 2 - Frontend
5 cd frontend && npm run dev
6
7 # Terminal 3 - ngrok (optional, for public access)
8 ngrok http 8080
```

部署前端:

```
1 cd frontend
2 npm run build      # Vite 打包到 dist/
3 npm run deploy     # gh-pages 推送至 GitHub Pages
```

环境变量 (TURN 功能需要):

```
1 set TWILIO_ACCOUNT_SID=ACxxxxxxxxxxx
2 set TWILIO_AUTH_TOKEN=your_auth_token
3 cd server && npm run dev
```

7.3 Data Flow: A Complete Game Round

玩家 A 打开页面

onMounted → restoreAvatar → BGM 默认关闭

玩家 A 创建房间

connect() → create_room → server: new GameRoom

A 收到 room_created (roomId)

URL 写入 ?auto_join=xxxx&name=AAA

玩家 B/C/D 加入房间

join_room(roomId) → server: addPlayer

room_players 广播 → 4人列表显示

4人 ready → 服务器开局

```
GameRoom.startGame()  
    Fisher-Yates 洗牌 108 张  
    掷骰子 (d1,d2) → 定庄家起手位置  
    扳王 → calculateWang(diTile)  
    发牌 53 张 (庄家14张, 闲家13张)  
    game_start 广播 (每人只收到自己的 hand)
```

游戏循环:

A 出牌 → send({type:'discard',tile})
→ server: handleDiscard → 移除手牌 → broadcastPublic
→ 其他人收到 game_state: 更新弃牌池 + 手牌数

checkIntercepts(A→B/C/D):
 有人能胡 → endRound(winner)
 有人能碰/杠 → action_prompt (等待选择)
 下家能吃 → action_prompt (等待选择)
 无人拦截 → nextTurn → B 摸牌 → broadcastPublic

B 摸牌 → 自摸检测 → 出牌 → 循环继续
超时保护: 10s 无响应 → 强制跳过

胡牌 → endRound(winnerIndex)
 赢家坐庄 (dealerIndex = winnerIndex)
 亮牌 showdownHands (四家手牌全部可见)
 各玩家点击"确认继续" → playerReadyForNext
 4人全部确认 → startGame (下一局)

Future Work

- **Mahjong Agent:** 基于深度强化学习 (PPO/DQN) + MCTS 自我对弈, 训练达到人类高手水平的 AI。
- **语音升级:** 引入 SFU 中转替代 P2P 网状, 解决 4 人语音不稳定问题。
- **音效录制:** 完成 34 个预录制 mp3 文件, 提升游戏沉浸感。
- **观战增强:** 支持多视角自动切换、回放功能。
- **性能优化:** App.vue 分拆为多个组件, 降低单文件复杂度。
- **移动端原生体验:** PWA 离线支持、推送通知。

Code Availability

本项目完全开源，代码仓库：<https://github.com/JaijaiC/TJMJ>

欢迎 Fork / Star / Pull Request。

Disclaimer

本项目为个人娱乐学习用途，不涉及任何商业行为。所有规则及实现仅供参考，实际游玩请以当地线下规则为准。因使用本软件产生的任何争议或损失，开发者不承担相关责任。

Acknowledgments

感谢湖南桃江的家人朋友提供规则校验与测试反馈；感谢开源社区 (Vue.js, Node.js, Vite, ws) 提供的优秀工具链。

参考项目：

- OpenMajiang (<https://gitee.com/mirrors/OpenMajiang>)
- 武汉麻将在线 (<http://cdn0001.afrxvk.cn/whmj/go.html>)